

PROJET NATACHA — ARCHITECTURE RÉSEAU

Cartographie Documentaire des Flux, Machines et Ports de Communication

Auteur : Thierry VIEIL • Version : 1.0 • Date : 16 Mai 2026 • Environnement : Écosystème Distribué (ARM64 / x86_64)

Note d'architecture : Le Projet Natacha repose sur une philosophie de découplage matériel et logiciel (système distribué). Afin d'optimiser l'utilisation des ressources énergétiques et de calcul, les tâches de réflexion, de capture auriculaire, de synthèse vocale et d'accès à la connaissance locale sont réparties sur différents nœuds communicant par protocoles IP.

1. Synthèse Globale des Flux Réseau

Pour assurer la modularité totale du système, trois entités physiques principales (ou rôles réseaux) interagissent en permanence. Les flux se divisent en deux grandes catégories : les messages asynchrones à faible latence (MQTT) et les flux de streaming brut en temps réel (GStreamer UDP).

Service / Application	Port Réseau	Protocole	Machine Serveur / Hôte	Rôle Réseau & Intervenants
Mosquitto (MQTT Broker)	1883	TCP	Cerveau (192.168.1.100)	Le cœur des messages. Centralise toutes les publications et lectures.
GStreamer (Audio Stream)	5000	UDP	Oreille / Ryzen (192.168.1.90)	Flux audio PCM brut streaming de la Bouche vers l'émetteur final.
Llama.cpp (API Serveur IA)	8080	TCP	Cerveau (192.168.1.100)	Héberge le grand modèle de langage local pour la génération des réponses.
Kiwix (Base de Connaissances)	8000	TCP	Cerveau (192.168.1.100)	Serveur HTTP local distribuant le savoir hors-ligne (Wikipédia, etc.).

2. Focus Topologique : Le Protocole Mosquitto (MQTT)

Le protocole MQTT fonctionne selon un modèle de type *Publication / Abonnement*. Il requiert un Broker centralisé qui reçoit tous les messages et les distribue aux clients abonnés aux thématiques (topics) spécifiques.

- **Le Serveur (Broker)** : S'exécute sur le **Mini-PC Cerveau** (`192.168.1.100`). Il écoute en permanence sur le port standard **1883** .
- **La Machine qui publie (Publisher)** : Le nœud qui génère une information. Par exemple, le Cerveau publie la réponse textuelle générée par l'IA sur le topic `natacha/reponse`.
- **La Machine qui lit (Subscriber)** : La carte **Orange Pi 6 Plus (Bouche)** (`192.168.1.X`) est abonnée au topic `natacha/reponse`. Dès qu'un message y est déposé par le serveur, la Bouche capte l'information textuelle pour déclencher immédiatement la synthèse vocale Piper.

3. Le Flux Audio Temps Réel : GStreamer

Contrairement à MQTT qui transmet des chaînes de caractères courtes sous TCP, la voix synthétisée requiert un transport ultra-rapide sans accusé de réception pour éviter toute saccade. C'est l'utilité du protocole UDP via GStreamer.

- **L'Émetteur (Source)** : L'**Orange Pi 6 Plus (Bouche)** génère le fichier WAV local via Piper à une fréquence d'échantillonnage de `f_s = 22050 ext{ Hz}`. La commande `gst-launch-1.0` encapsule le flux brut (format S16LE, monocal) et l'expédie vers le réseau.
- **Le Récepteur (Sink)** : Le flux est poussé directement vers l'adresse IP de l'**Oreille (Mini PC Ryzen)** (`192.168.1.90`) sur le port UDP **5000** . Le Ryzen maintient un pipeline d'écoute passif qui redirige instantanément les paquets reçus vers la carte son physique.

4. Services Internes et Moteurs de Réflexion (Llama.cpp & Kiwix)

Ces services sont purement orientés "données" et s'exécutent au cœur du nœud le plus puissant techniquement (le Cerveau) :

- **Llama.cpp (Port 8080)** : Expose une API HTTP compatible OpenAI. Le script maître interroge ce port en local (ou depuis un nœud périphérique) pour soumettre le texte capté par l'oreille et récupérer la prédiction textuelle de l'IA.
- **Kiwix (Port 8000)** : Agit comme une bibliothèque interne. En interrogeant ce port via des requêtes HTTP, le Cerveau peut fouiller de manière totalement autonome dans des fichiers compressés ZIM (contenant l'intégralité de Wikipédia ou de guides techniques) sans aucune connexion à Internet.